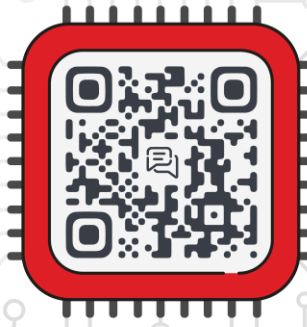




نظري

5



18



2160 sp

كلية الهندسة المعلوماتية

السنة الثالثة

HTML5 Forms + Intro to javascript

د. غاندي هسام

Programming Languages



RB Informatics; 12/11/2025

لغات البرمجة

سوف نكمل في chapter 5 في الـ HTML و خاصة الـ forms و قسم الـ table من المرجع غير داخل.
و ثم ننتقل إلى chapter 8 لنكمل في الـ JavaScript والذي يحوي أساسيات JS.

مقدمة عن النماذج (forms):

الـ forms في HTML هي الطريقة الأساسية التي تمكن المستخدم من ارسال معلومات إلى السيرفر، وبالتالي فهي وسيلة تفاعل بين المستخدم و الموقع.

- **Forms** توفر للمستخدم طريقة للتفاعل مع السيرفر ، اي بدلاً من كون الصفحة مجرد محتوى يُعرض فقط ، بإمكان المستخدم ادخال معلومات يتم ارسالها الى السيرفر لمعالجتها.
- باستخدام الـ form يمكن للمستخدم ادخال نص (مثل الاسم ، الايميل ، كلمة المرور) ، اختيار عناصر من قوائم الضغط على الأزرار، تحديد خيارات مثل **checkbox** أو **radio**.
- ماذا يفعل السيرفر بالبيانات ؟
 - حفظ البيانات في قاعدة بيانات.
 - التحقق من صحة المعلومات.
 - التواصل مع خدمات خارجية.
 - انشاء صفحة HTML جديدة حسب المدخلات.

مثال: تسجيل دخول، تسجيل حساب، ارسال رسالة، إضافة منتج...

- في HTML5 تم تطوير الفورمات و اضافوا:
 - عناصر إدخال جديدة (color,date,number,email....)
 - خيارات تخصيص اكثر للحقول الموجودة سابقا.
- **الهدف:** جعل الفورم اسهل للمستخدم ، و اسهل للمبرمج ، و اكثر دقة بالسلوك.



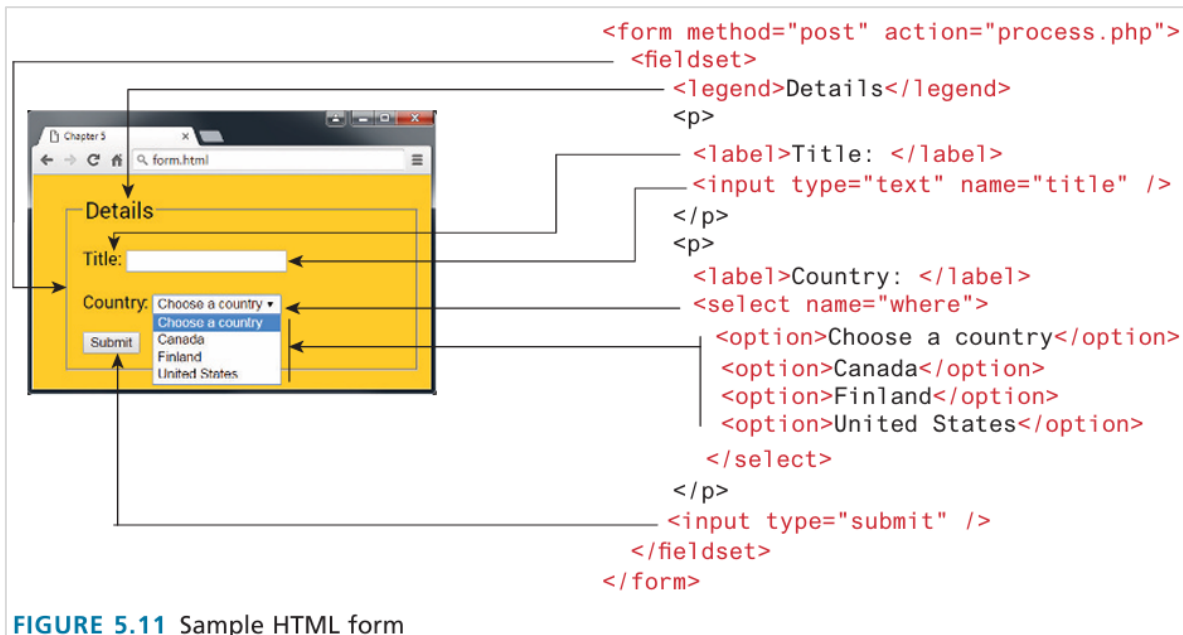


FIGURE 5.11 Sample HTML form

الشرح:

1. وسم <form>: نموذج إرسال

- يحتوي الخاصية "method='post'" وهي الطريقة التي ترسل بها البيانات للسيرفر.
- الخاصية "action='process.php'" تحدد الملف الذي سيستقبل البيانات ويعالجها.

2. وسم <fieldset>

- يُستخدم لتجميع حقول النموذج ضمن إطار واحد.
- يعطي شكل متناسق ومنظم للنموذج.

3. وسم <legend>

- عنوان المجموعة داخل الـ fieldset ، في المثال مكتوب "Details"

4. وسم <label>

- يُستخدم لكتابة عنوان الحقل.
- وجوده مهم لأنه يوضح للمستخدم وظيفة الحقل.
- أيضاً يساعد في الوصول (Accessibility).

5. وسم <input type='text'>

- حقل إدخال نص عادي.
- الخاصية "name='title'" تعطي اسماً للبيانات عند إرسالها للسيرفر.

6. وسم <select>

- يُستخدم لإنشاء قائمة منسدلة.



○ يحتوي الخاصية `name="where"` والتي تمثل اسم القيمة المرسلة للسيرفر.

7. وسوم `<option>`

○ تمثل الخيارات الموجودة داخل القائمة المنسدلة.

○ في المثال يوجد: "Choose a country — Canada — Finland — United States"

8. وسوم `<input type="submit">`

○ الزر الذي يرسل النموذج عند الضغط عليه ، عنوان الزر الافتراضي هو "Submit"

⚡ كيف تعمل نماذج (forms) HTML:

Form لوحده لا يفعل شيء، هو فقط وسيلة لجمع البيانات، لكن معالجة البيانات تتم دائماً عن طريق السيرفر او JavaScript بعد الضغط على submit.

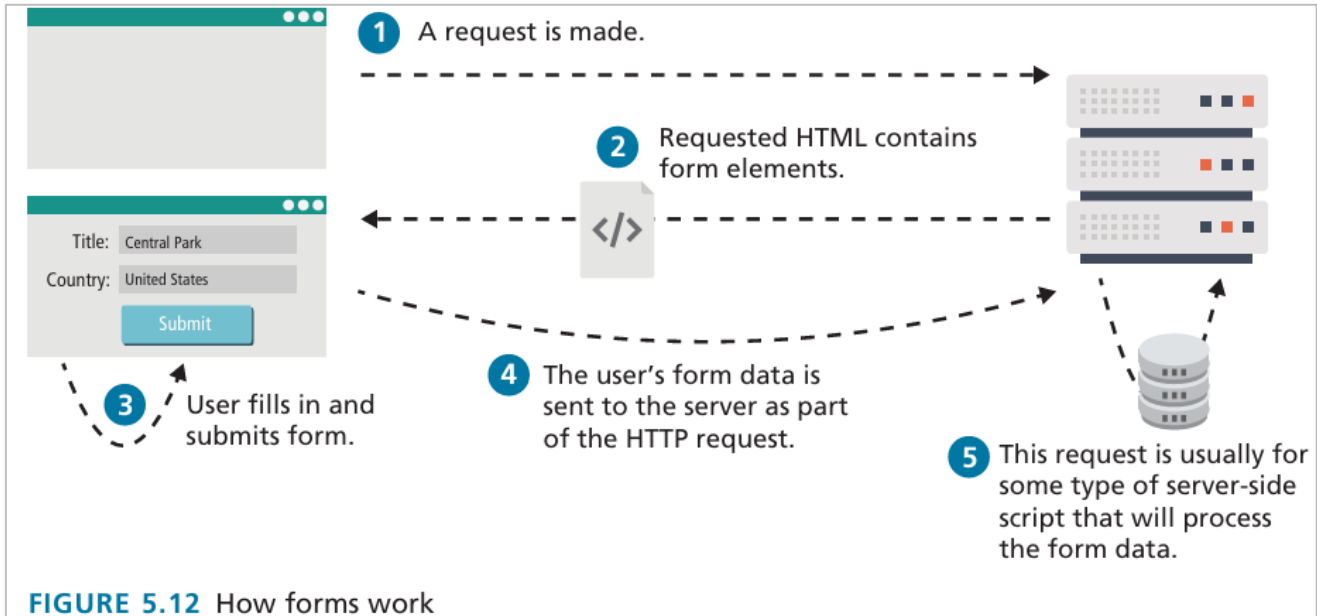


FIGURE 5.12 How forms work

1. A request is made

المتصفح يرسل طلب (HTTP Request) إلى السيرفر لجلب صفحة HTML

2. Requested HTML contains form elements

السيرفر يرسل صفحة HTML ، وهذه الصفحة تحتوي على عناصر form مثل `<form>`, `<input>`, `<label>`, `<select>`

3. User fills in and submits form

المستخدم يكتب البيانات داخل الـ form (مثل العنوان والدولة)، ثم يضغط زر Submit

4. The user's form data is sent to the server

عند الضغط على Submit، المتصفح يرسل بيانات النموذج (form data) إلى السيرفر ضمن طلب HTTP جديد

5. Server processes the form data

الطلب (الذي يحتوي معلومات المستخدم) يصل إلى السيرفر، ويقوم السيرفر بمعالجة البيانات باستخدام سكربت server-side مثل: PHP-Node.js-Python-Java.

ملاحظة: JavaScript تستطيع ارسال البيانات بدون زر submit.

Query strings ↩

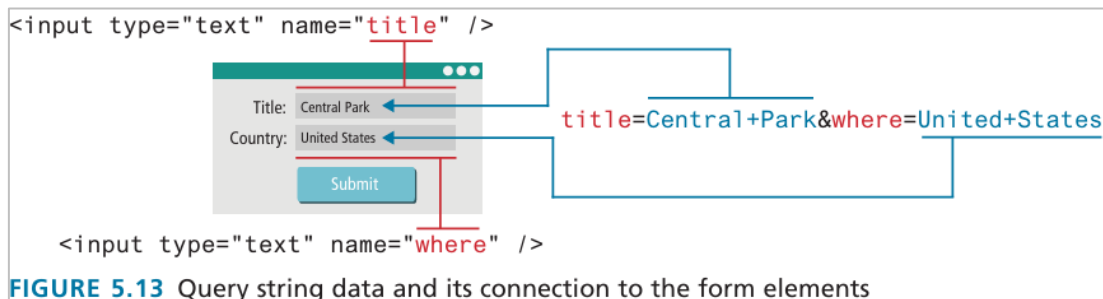
قد تتساءل كيف يرسل المتصفح بيانات النموذج (Form) إلى السيرفر؟

المتصفح لا يرسل البيانات مباشرة، بل يجمعها داخل شيء اسمه query string

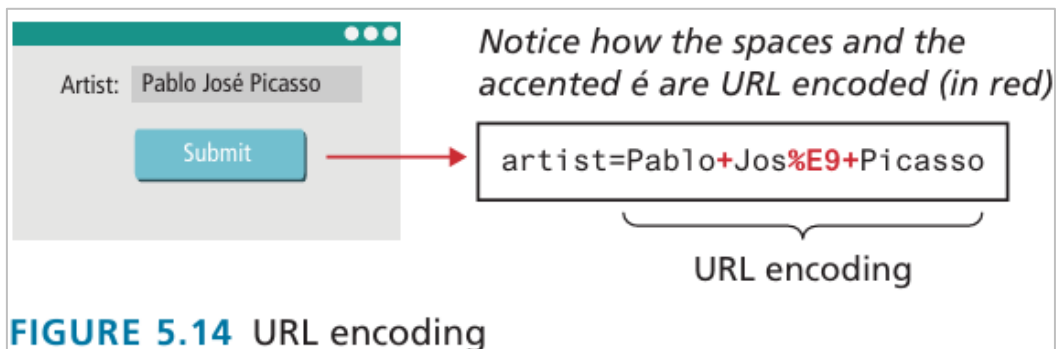
- المتصفح يقوم بتجميع بيانات المستخدم داخل صيغة معينة تسمى **Query String** وهي عبارة عن طريقة لتمثيل البيانات مع رابط الصفحة (URL)
- الـ Query String عبارة عن سلسلة من الأزواج (name=value) كل جزء يحتوي اسم الحقل = القيمة التي أدخلها المستخدم وبين كل زوج وزوج يتم الفصل باستخدام الرمز: **& (ampersand)**
- الأسماء المستخدمة داخل الـ Query String مصدرها الـ HTML Form يعني اسم الحقل الذي يظهر داخل الـ Query String هو نفسه قيمة الـ name attribute في عنصر الإدخال (input / select)
- كل عنصر داخل الـ Form يجب أن يحتوي على name attribute لأن هذا الاسم هو الذي يظهر داخل الـ Query String ويمثل المفتاح الذي ترسل به البيانات إلى السيرفر.

مثال 1:

هذا المثال يوضح كيف تتحول المدخلات التي يكتبها المستخدم داخل الفورم إلى بيانات داخل الـ URL عند الإرسال. نلاحظ أن المسافة تتحول لـ **+**.



مثال 2:



نلاحظ أن الحرف é يتحول إلى **%E9**

artist=Pablo+Jos%E9+Picasso وهذا الترميز يسمى **URL encoding**.

:The <form> Element

الفرم هو نموذج ادخال بيانات ، يحتوي صفتين مهمتين جدا ، هذان ال attributes اساسيان في أي form في HTML:

1. صفة action :

توضح الرابط URL الخاص بالملف او السيرفر الذي سيستقبل بيانات ال form و يقوم بمعالجتها.

مثال: `<form action="process.php">`

يعني أن البيانات سيتم إرسالها الى ملف process.php ليقوم بمعالجتها.

2. صفة method:

○ تحدد كيف سيتم ارسال البيانات من المتصفح إلى السيرفر.

○ يوجد خياران فقط GET, POST

○ الفرق بين GET و POST

⬇ GET:

يرسل البيانات ضمن عنوان الرابط URL، يعني تقوم المتصفحات بوضع query string داخل الرابط و تكون ظاهرة للمستخدم.

⬇ POST:

لا يرسل البيانات داخل ال URL ، يتم وضع البيانات داخل HTTP Header بعد ما يسمى HTTP variables ، لذلك يُستخدم غالبا لإرسال البيانات الحساسة أو الكبيرة.

مثال:

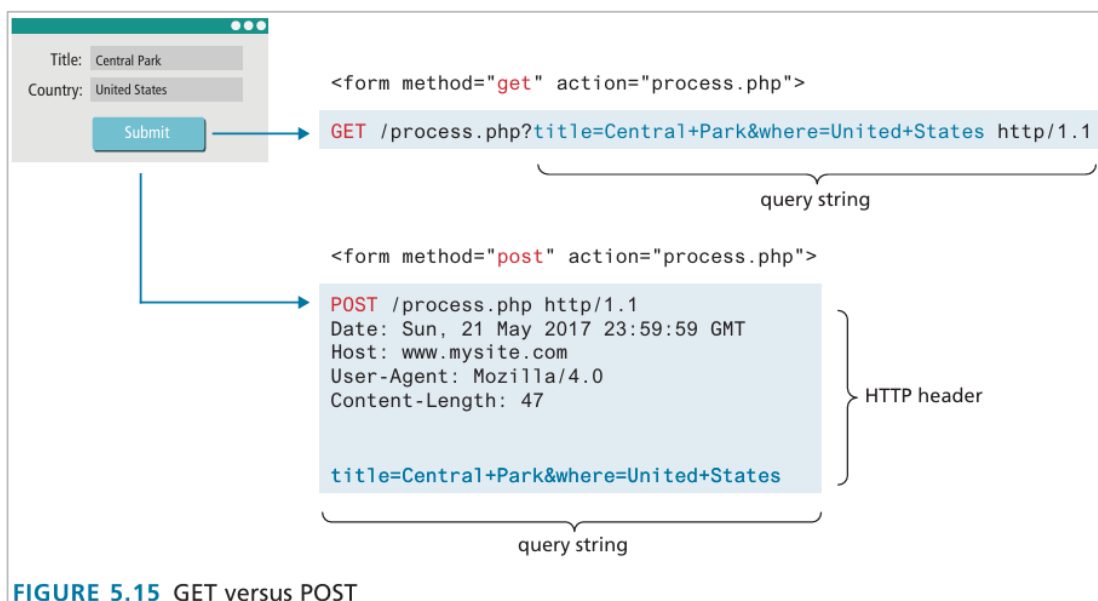


FIGURE 5.15 GET versus POST

❖ أول مثال في الصورة (GET):

عندما يضغط المستخدم submit <= يتم إضافة البيانات إلى نهاية الـ URL كـ query string

ملاحظات مهمة:

القيم تُرسل ضمن الـ URL.

يمكن رؤيتها بوضوح في شريط العنوان.

تبقى محفوظة في History و cache.

مثال: عند تسجيل الدخول لموقع في الموبايل مثلاً يتم حفظ كلمة المرور مثلاً، و عند التسجيل مرة ثانية، يتم ملأ الخانات تلقائياً من النسخة المحفوظة، و هذا شيء مفيد، ولكن إذا كان الموبايل ذاته مع شخص آخر، و عند التسجيل بموقع، ستتم نفس العملية، مما يؤدي إلى انتحال شخصية و هذا شيء سلبي.



يمكن عمل Bookmark لها.

يوجد حد لعدد محارف البيانات.

ليست مناسبة للبيانات الحساسة.

❖ ثاني مثال في الصورة (POST):

عند الضغط على زر submit <= البيانات لا تظهر داخل الـ URL ، و يتم إرسالها داخل HTTP Header .

ملاحظات مهمة:

البيانات مخفية عن المستخدم

يمكن إرسال بيانات كبيرة

يمكن إرسال Binary Data (ملفات و صور)

لا تُحفظ في History أو Bookmarks

تُعتبر أفضل للبيانات الحساسة.

لا حدود عملية للحجم.

Type	Advantages and Disadvantages
GET	Data can be clearly seen in the address bar. This may be an advantage during development but a disadvantage in production. Data remains in browser history and cache. Again this may be beneficial to some users, but it is a security risk on public computers. Data can be bookmarked (also an advantage and a disadvantage). There is a limit on the number of characters in the returned form data.
POST	Data can contain binary data. Data is hidden from user. Submitted data is not stored in cache, history, or bookmarks.

TABLE 5.1 GET versus POST

هذا الجدول يعرض أهم العناصر الخاصة بالفورم بـ HTML5. كل عنصر له وظيفة معينة داخل النموذج:

Type	Description
<button>	Defines a clickable button.
<datalist>	An HTML5 element that defines lists of pre-defined values to use with input fields.
<fieldset>	Groups related elements in a form together.
<form>	Defines the form container.
<input>	Defines an input field. HTML5 defines over 20 different types of input.
<label>	Defines a label for a form input element.
<legend>	Defines the label for a fieldset group.
<optgroup>	Defines a group of related options in a multi-item list.
<option>	Defines an option in a multi-item list.
<output>	Defines the result of a calculation.
<select>	Defines a multi-item list.
<textarea>	Defines a multiline text entry box.

TABLE 5.2 Form-Related HTML Elements

1. Text input controls:

تستخدم النماذج عادة لجمع معلومات نصية من المستخدم ، مثل صندوق البحث أو التسجيل أو تسجيل الدخول .. لذلك نحتاج لأنواع مختلفة من الحقول النصية.

عنصر **<input>** هو أهم عنصر داخل الـ form ، لأنه المسؤول عن **أخذ البيانات** من المستخدم. هذا العنصر له نوع **type** يحدد شكل وطريقة الإدخال.

Type	Description
email	Creates a single-line text entry box suitable for entering an email address. This is an HTML5 element. Some devices (such as the iPhone) will provide a specialized keyboard for this element. Some browsers will perform validation when form is submitted. <input type="email" ... />
password	Creates a single-line text entry box for a password (which masks the user entry as bullets or some other character) <input type="password" ... />
search	Creates a single-line text entry box suitable for a search string. This is an HTML5 element. Some browsers on some platforms will style search elements differently or will provide a clear field icon within the text box. <input type="search" ... />
text	Creates a single-line text entry box. <input type="text" name="title" />
textarea	Creates a multiline text entry box. You can add content text or if using an HTML5 browser, placeholder text (hint text that disappears once user begins typing into the field). <textarea rows="3" ... />

أنواع حقول النص:

Email

- ينشئ صندوق نص مخصص لإدخال بريد إلكتروني.
- بعض الأجهزة مثل iPhone تعرض لوحة مفاتيح خاصة للبريد.

○ بعض المتصفحات تجري عملية تحقق (validation) عند إرسال النموذج. صيغة الكتابة في الجدول السابق.

❖ Password:

- ينشئ صندوق نص لإدخال كلمة مرور .
- المتصفح يخفي الأحرف (باستخدام نقاط او نجوم) .

❖ Search:

- ينشئ صندوق نص مخصص للبحث.
- على بعض المتصفحات يتم تنسيق الحقل بشكل مختلف، وقد يظهر زر لمسح النص.

❖ Text:

- ينشئ صندوق نصي عادي مكون من سطر واحد.

❖ Textarea:

- ينشئ صندوق نص يحتوي عدة أسطر (متعدد الأسطر).
- يمكن وضع نص مبدئي placeholder في متصفح HTML5 .

ملاحظة: رغم أن بعض عناصر النص في HTML5 ليست مدعومة بشكل كامل في جميع المتصفحات ، إلا أنها تعمل كحقول نص عادية في المتصفحات القديمة.

2. Select lists

❖ ما هو عنصر <select>:

عنصر <select> يُستخدم لإنشاء قائمة يمكن للمستخدم اختيار item واحد منها أو أكثر (حسب الإعدادات) .

❖ ال options داخل ال <select>:

الخيارات داخل القائمة تُعرّف باستخدام عنصر <option>.

يمكن عرض الخيارات بطريقتين: Drop-down list (قائمة منسدلة – مخفية حتى ينقر المستخدم عليها) ، Multiple rows list (قائمة طويلة ظاهرة ، أكثر من سطر).

❖ Grouping عبر <optgroup>:

يمكن تجميع ال options ضمن مجموعات باستخدام عنصر <optgroup> هذا يفيد مثلاً في تجميع دول حسب القارة، أو فئات حسب القسم.

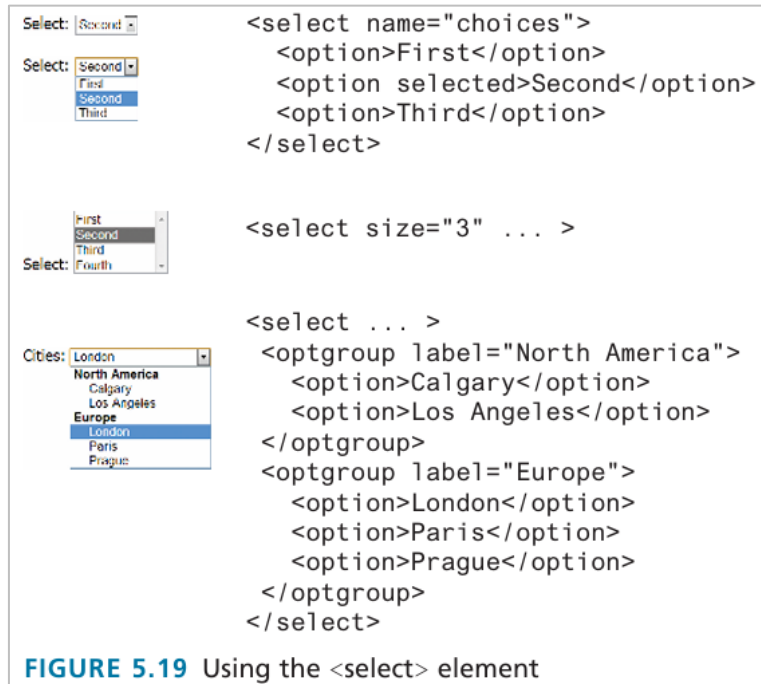
❖ خاصية selected:

الخاصية selected داخل <option> تجعل هذا الخيار هو الخيار الافتراضي عند ظهور الصفحة.

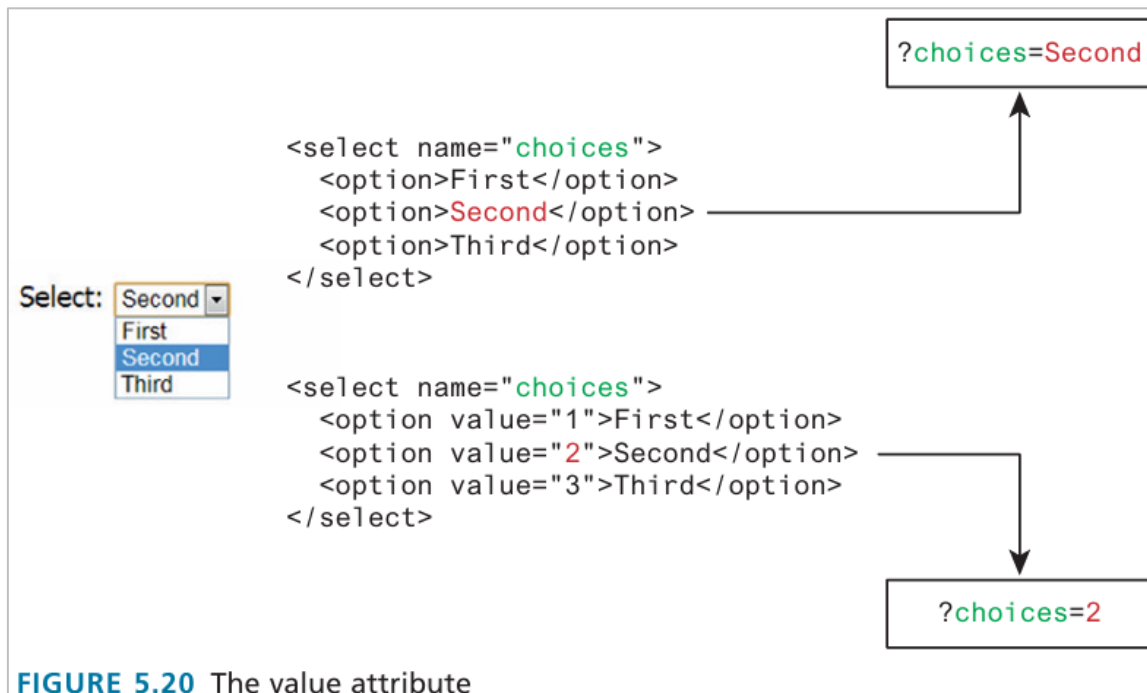
❖ خاصية value:

الخاصية value داخل <option> تحدد القيمة التي سيتم إرسالها للسيرفر عند اختيار هذا الخيار.

يعني الذي يظهر للمستخدم يختلف عن الذي يُرسل للسيرفر أحياناً.
مثال 1:



مثال 2:



3. Radio Buttons

نستخدم Radio Buttons عندما نريد من المستخدم ان يختار خيار واحد فقط من مجموعة خيارات صغيرة، و كل الخيارات تكون ظاهرة امام المستخدم بذات الوقت (أي ليست Dropdown).

Continent:

☐ North America☒ South America☐ Asia☐North America
☒

☐Asia

FIGURE 5.21 Radio buttons

شرح المثال:

- **Type="radio"**: تعني هذا الحقل عبارة عن زر اختيار فردي (radio buttons)
- **Name ="where"**: جميع العناصر لها نفس الاسم "where" ، مما يؤدي لجعلهن ضمن مجموعة واحدة و بالتالي المستخدم يمكنه اختيار خيار واحد فقط بينها.
- **Value ="1" , "2" , "3"**: هذا هو الرقم (القيمة) الذي يُرسل للسيرفر عندما المستخدم يضغط submit.
- **Checked**: هذا الخيار يظهر خيار محدد مسبقا (south America هي الخيار المحدد افتراضياً).
- **Checkbox**: هو مربع صغير يسمح للمستخدم يختار اكثر من خيار بنفس الوقت (على عكس ال radio الذي يسمح بخيار واحد فقط)

أمثلة:

I accept the software license ☒ `<label>I accept the software license</label>`
`<input type="checkbox" name="accept" >`

Where would you like to visit? `<label>Where would you like to visit? </label><br/>`
☒ Canada `<input type="checkbox" name="visit" value="canada">Canada<br/>`
☐ France `<input type="checkbox" name="visit" value="france">France<br/>`
☒ Germany `<input type="checkbox" name="visit" value="germany">Germany`

`?accept=on&visit=canada&visit=germany`

FIGURE 5.22 Checkbox buttons

شرح المثال الأول: عندما المستخدم يعلّم على المربع، سوف يرسل المتصفح `accept =on` وإذا غير محدد تكون `off` إذا كانت ال `value` غير محدد.

و لكن في هذا المثال سوف يرسل `visit="Canada"` و حسب الاختيارات.

f /ITE.RBCs

10

4. Button:

يتم استخدام الأزرار لتنفيذ عمليات في النموذج مثل الإرسال أو التصفير أو تنفيذ كود معين .
أنواع الأزرار:

Type	Description
<code><input type="submit"></code>	Creates a button that submits the form data to the server.
<code><input type="reset"></code>	Creates a button that clears any of the user's already entered form data.
<code><input type="button"></code>	Creates a custom button. This button may require JavaScript for it to actually perform any action.
<code><input type="image"></code>	Creates a custom submit button that uses an image for its display.
<code><button></code>	Creates a custom button. The <code><button></code> element differs from <code><input type="button"></code> in that you can completely customize what appears in the button; using it, you can, for instance, include both images and text, or skip server-side processing entirely by using hyperlinks. You can turn the button into a submit button by using the <code>type="submit"</code> attribute.

TABLE 5.4 Button Elements

Type	Description
Submit button	يرسل بيانات النموذج الى السيرفر
Reset button	يمسح كل البيانات التي ادخلها المستخدم و يعيد الحقول للوضع الأصلي .
Button	زر مخصص ، لن يعمل بدون JavaScript (لا يرسل فورم و لا يمسح)
Image button	زر ارسال لكن شكله عبارة عن صورة بدلا من زر عادي
<code><button></code> element	وسم HTML كامل ، يمكنك وضع بداخله نص+صورة +رابط ... يمكن تحويله لزر ارسال اذا كتبنا <code><button type = "submit"></code>

أمثلة:

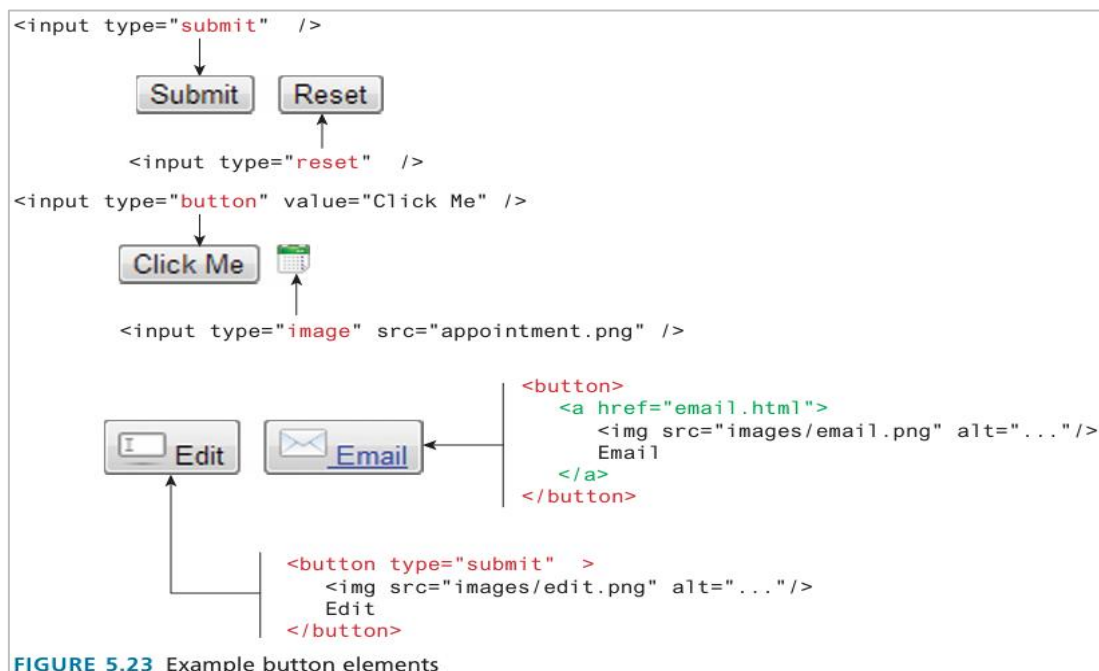


FIGURE 5.23 Example button elements

Validating user input (التحقق من ادخال المستخدم):

الهدف من التحقق هو التأكد ان البيانات التي كتبها المستخدم صحيحة، منطقية، و مسموح استخدامها قبل ارسالها للسيرفر.

Type of input validation: ⚡

- Required information (الحقول الإلزامية):
 - وهي حقول ادخال المعلومات الأساسية، و لا يمكن تركها فارغة مثل البريد أو كلمة المرور.
- Correct data Type (نوع البيانات الصحيح):
 - مثلاً: الحقل يجب ان يكون رقم ، او بريد الكتروني أو نص.
- Correct format (التنسيق الصحيح):
 - مثل تنسيق الايميل example@gmail.com
- Comparison (المقارنة):
 - مقارنة قيمتين مع بعض ، مثل : تأكيد كلمة المرور ، تاريخ النهاية اكبر من تاريخ البداية.
- Rang check (نطاق القيم):
 - مثل: العمر بين 18 و 60 او العلامة بين 0 و 100
- Custom (تحقق خاص):
 - للأمور المعقدة التي تتغير حسب التطبيق، مثل: رقم وطني، صيغة خاصة رقمية أو نصية، كلمات مرور قوية.

Notify the user (اعلام المستخدم بالخطأ): ⚡

عندما يرتكب المستخدم خطأ معين يجب اخباره:

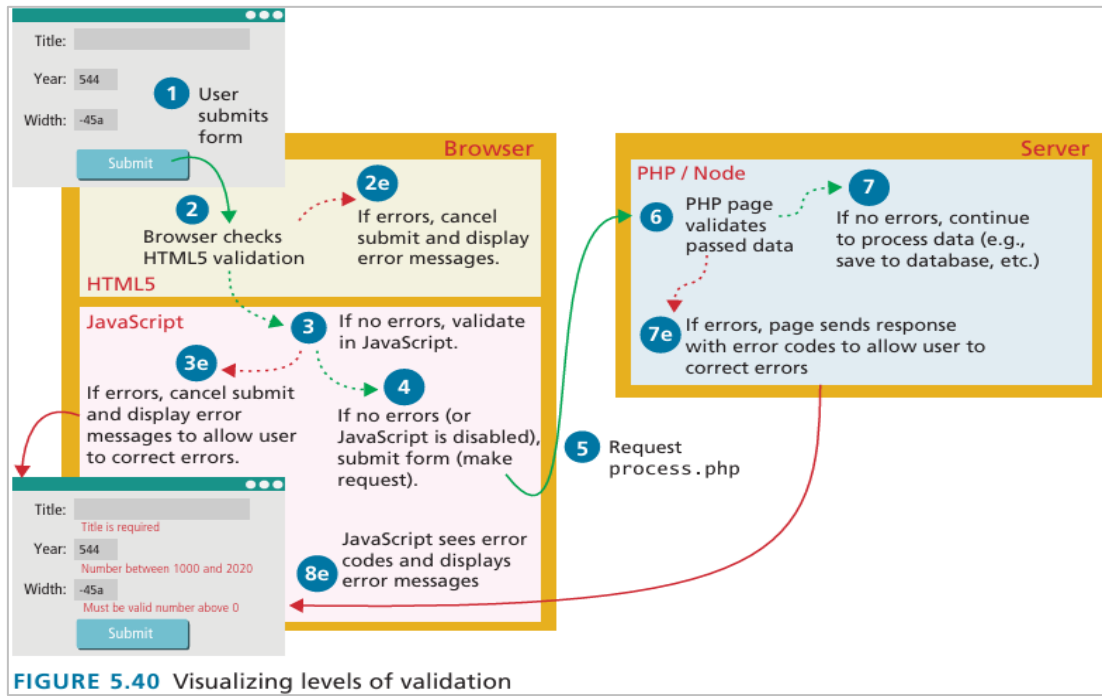
- What is the problem?
- Where is the problem?
- If appropriate, how do I fix it ?

How to reduce validation Errors ⚡

- استخدام رسائل JavaScript ، مثل Alerts او رسائل منبثقة للتنبيه .
- إضافة Hints داخل النموذج مثل: "enter your email" = placeholder
- إعطاء تلميح للمستخدم قبل بما يجب كتابته.
- Popovers أو Tooltips: رسائل صغيرة تظهر عند وضع المؤشر على الحقل



مستويات التحقق في فورم HTML:



لدينا 3 مستويات: Browser , JavaScript, Server

JavaScript

JavaScript لغة كائنية التوجه (object-oriented)، أي كل شي داخلها عبارة عن كائن (object)، حتى المتغيرات تعتبر كائنات لأنها تملك خصائص (properties) و دوال (methods).

❖ الدوال functions نفسها تعامل ككائنات أيضاً، عكس اللغات مثل java و c# و c++.

JS لا تعتمد على ال classes التقليدية مثل java...، بل تستخدم نظام prototype-Based لتوليد الكائنات (يعني نصي و ليس binary)

❖ JS ديناميكية الأنواع (Dynamically Typed)، مما يعني ان نوع المتغير يمكن أن يتغير تلقائياً حسب القيمة (لان تحديد النمط يحدث بال Runtime، و ليس بال compile time مثل: java و c).

آلية عمل JS في المتصفح و تطوره للتعامل مع web APIs:

- كيف يتم تحميل و تنفيذ ملف JS في المتصفح ؟
- يقوم المستخدم بطلب صفحة HTML، المتصفح يرسل Request للسيرفر لي جلب ملف ال HTML.
- ملف HTML الذي وصل للمتصفح يحتوي على وسم يطلب ملف JavaScript خارجي.
- **مثل:** `<script src="app.js"></script>`، فالمتصفح يفهم أنه يجب ان يطلب ملف JS أيضاً.
- المتصفح يرسل طلب جديد للسيرفر للحصول على ملف JavaScript request إضافي لملف ال JS.
- السيرفر يرسل ملف JavaScript إلى المتصفح، الملف يرجع على شكل نص JavaScript.

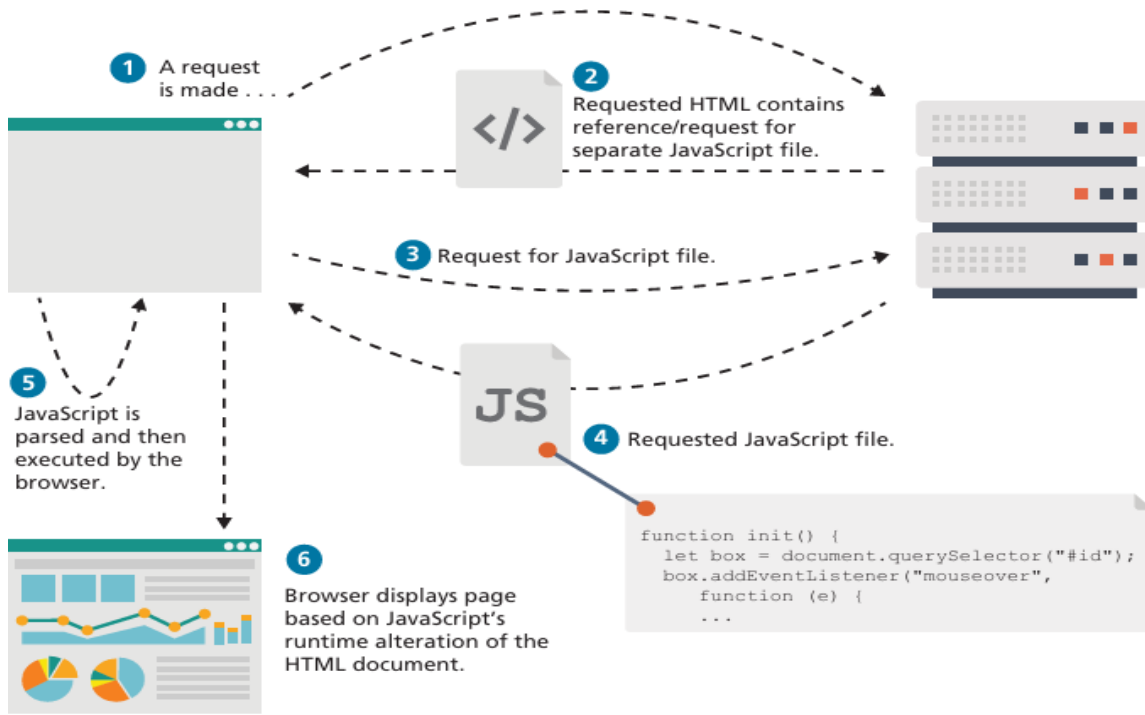


FIGURE 8.1 Downloading and executing a client-side JavaScript script

- المتصفح يقرأ (parse) الكود وينفّذه ، أي تعليمات داخل ملف JS يتم تنفيذها داخل المتصفح.
- بعد تنفيذ JavaScript ، المتصفح يحدّث الصفحة حسب التغييرات التي أجراها الكود على الـ HTML/CSS ، مثل تغيير نص ، إضافة نص ، تعديل الألوان...

أسلوب عمل JS الحديث (التواصل مع web APIs):

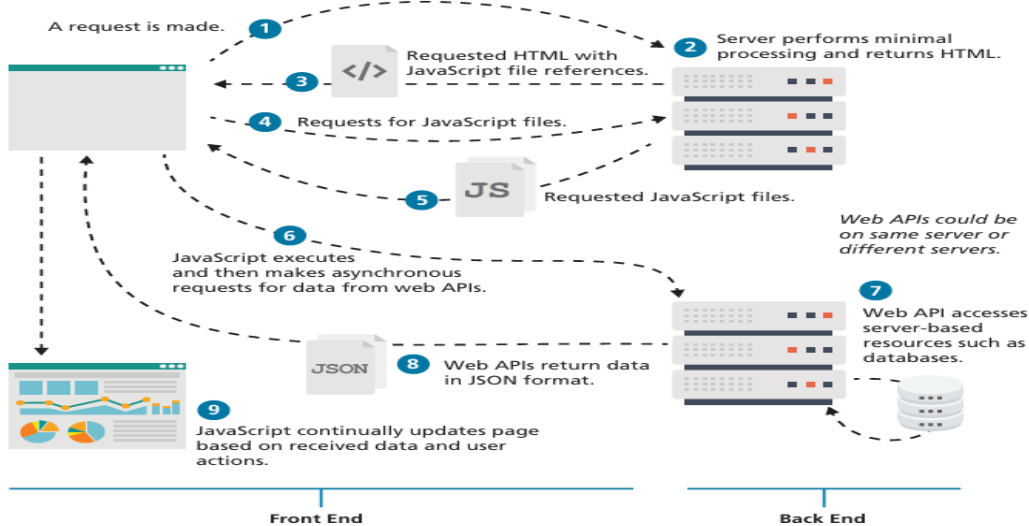


FIGURE 8.2 Contemporary JavaScript coding

- المستخدم يرسل طلب لصفحة الويب ، المتصفح يطلب ملف HTML من السيرفر.
- السيرفر يقوم بمعالجة بسيطة ويرجع ملف HTML ، يحتوي على روابط لملفات JavaScript.
- المتصفح يحمل ملف HTML ويلاحظ وجود ملفات JavaScript داخله.
- المتصفح يرسل طلبات جديدة للحصول على ملفات JavaScript Requests إضافية.

- السيرفر يرجع ملفات JavaScript المطلوبة و يتم إرسالها للمتصفح كنص JS.
- JavaScript يبدأ بالتنفيذ داخل المتصفح (Front-End) وبعد التنفيذ، يبدأ الكود بطلب بيانات من Web APIs بشكل Asynchronous.
- Web API تتصل بموارد في السيرفر مثل قواعد البيانات، هذه الـ API قد تكون على نفس السيرفر أو سيرفر آخر.
- Web API ترجع البيانات بصيغة JSON ، هذا التنسيق مناسب للمتصفحات و JavaScript ، قديماً كانت ترجع .xml
- JavaScript يقوم بتحديث الصفحة بشكل مستمر ، يحدث الواجهة حسب البيانات الواردة من الـ API وتفاعل المستخدم.

تساؤل: هل JavaScript تنحصر في الباك فقط، أو الفرونت أو الموبايل ؟ لا JavaScript تدخل في كل شي و تعتبر اكثر لغة مستخدمة.

استخدامات JavaScript خارج المتصفح:

- JavaScript داخل تطبيقات غير المتصفح
- JavaScript لم تعد مقتصرة على المتصفح فقط، أصبح يمكن تشغيلها على السيرفر باستخدام بيئات تشغيل مثل Node.js، و هذا يجعلها تستخدم كلغة برمجة للسيرفر.
- قواعد بيانات جديدة تستخدم JavaScript
- بعض قواعد البيانات غير العلائقية (قواعد بيانات تعتمد على العلاقات بين الجداول) مثل MongoDB تسمح باستخدام JavaScript كلغة للاستعلام و معالجة البيانات.
- تطبيقات سطح المكتب المعقدة
- برامج كبيرة مثل Adobe Creative Suite و OpenOffice تستخدم JavaScript كلغة سكريبت داخلية لتنفيذ مهام معينة داخل البرنامج.
- تشغيل Js على أجهزة الهاردوير
- أجهزة مثل Oculus Rift و Arduino و Raspberry Pi تحتوي على محركات JavaScript مدمجة تسمح بتشغيل السكريبت للتحكم بالهاردوير.

كيفية تضمين JavaScript داخل صفحات الويب بالطرق المختلفة:

- **Embedded JavaScript**: أن تكتب كود js داخل وسم <script> بنفس ملف HTML و ليس داخل العنصر.
- **External JavaScript**: هو أفضل طريقة، و فعلياً هي الطريقة الموصى بها، تكتب كود JS في ملف مستقل name.js و ثم تربطه بملف HTML.

- **Inline JavaScript**: أن تكتب كود js داخل HTML element نفسه.

ملاحظة: هذه الطريقة سيئة جدا و لا ينصح باستخدامها و تعتبر أسوأ من inline CSS، هذه الطريقة تؤدي إلى فوضى بالكود و يصعب تنظييمه و التعديل عليه.

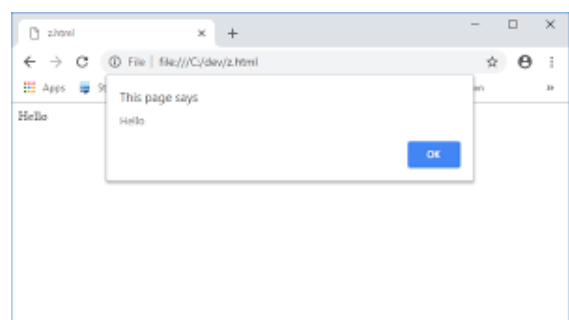
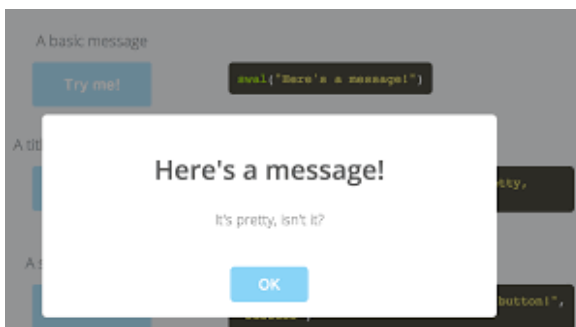
- **Pro Tip**: بعض المواقع الكبيرة و العالية الزيارات تستخدم embedded JavaScript و embedded CSS داخل صفحة HTML حتى تقلل عدد طلبات ال HTTP و بالتالي تسرع تحميل الصفحة ، لان الاكواد المضمنة عادةً حجمها صغير.
- اما في اغلب المواقع، يكون الاعتماد على الملفات الخارجية (External JS/CSS) افضل للأداء، لان المتصفح سيقوم بتخزين هذه الملفات (cache) و إعادة استخدامها عند زيارة صفحات أخرى من نفس الموقع ، مما يخفف التحميل و يحسن السرعة.
- بعد تضمين JavaScript في الصفحة، الخطوة التالية هي فهم كيف تعرض نتائج السكريبت عبر المتصفح كما يظهر لدينا بالجدول التالي:

```
let name = "Randy";
document.write("<h1>Title</h1>");
// this uses the concatenate operator (+)
document.write("Hello " + name + " and welcome");
```

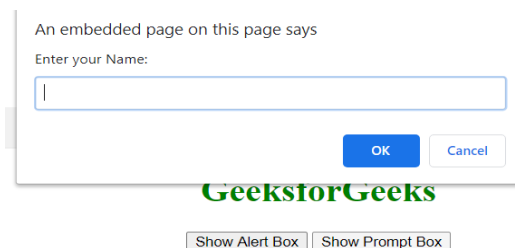
Method	Description
alert()	Displays content within a browser-controlled pop-up/modal window.
console.log()	Displays content in the browser's JavaScript console.
document.write()	Outputs the content (as markup) directly to the HTML document.
prompt()	Displays a message and an input field within a modal window.
confirm()	Displays a question in a modal window with ok and cancel buttons.

أمثلة توضيحية ارييسيزية:

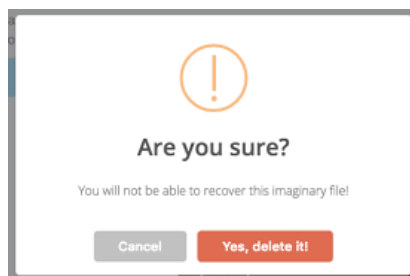
- ❖ alert(): نافذة تنبيه بسيطة تظهر للمستخدم مع زر فقط OK



- ❖ console.log(): المخرجات تظهر داخل console
- ❖ document.write(): يكتب مباشرة داخل الصفحة
- ❖ prompt(): نافذة فيها رسالة + مربع لإدخال النص



- ❖ confirm(): نافذة فيها OK و Cancel لتأكيد اختيار معين



مقارنة بسيطة:



Objects in JS:

في JS ، الـ object هو مجموعة من القيم (values) و كل قيمة لها اسم (name) و بينهم ' : ' .
ونسميها **Properties**.

🔗 كيف ننشئ object:

في JavaScript يوجد أكثر من طريقة لإنشاء objects ، لكن أشهر وأبسط طريقة هي: Object Literal Notation .
وهي كتابة الـ object مباشرة بين { } بهذا الشكل:

```
const photo = {
  id: 1,
  title: "Central Library",
  location: {
    country: "Canada",
    city: "Calgary"
  }
};
```

```
const objName = {
  name1: value1,
  name2: value2,
  // ...
  nameN: valueN
};
```

❖ يتم الوصول لل object الداخلي مثلاً: `photo.id` , `photo.location.city`

An object can contain ...

- primitive values
- array values
- other object literals
- arrays of objects

```
const country1 = {
  name: "Canada",
  languages: ["English", "French" ],
  capital: {
    name: "Ottawa",
    location: "45°24'N 75°40'W"
  },
  regions: [
    { name: "Ontario", capital: "Toronto" },
    { name: "Manitoba", capital: "Winnipeg" },
    { name: "Alberta", capital: "Edmonton" }
  ]
};
```

المثال يوضح ماذا يحوي ال object



انتهت المحاضرة

<The end>